Computación gráfica y ambientes virtuales (0494)

Práctico 09

Objetivo

El objetivo es mejorar la apariencia de la escena incorporando iluminación y efecto niebla. Por un lado, un sol que ilumine el terreno y la aeronave y por otro lado que la lejanía se funda con el horizonte del cielo por medio de la niebla. Hasta acá el color de cada punto de una superficie salía directamente de la textura o del color de la pieza, sin importar cómo estaba orientada esa superficie respecto de la luz, por lo que el relieve del terreno del Práctico 07, aunque está en la geometría, casi no se distingue en la pantalla. Con un modelo de iluminación, el color de cada fragmento pasa a depender de la normal de la superficie en ese punto, de la dirección de la luz y de la posición del observador (cámara). Agregando el efecto de niebla, el color de la pieza se mezcla con un porcentaje de color del horizonte función de la distancia al observador logrando eliminar el borde duro de terreno lo que aporta profundidad a la escena. Por último, se incorpora la corrección *gamma* para que los cálculos de iluminación se hagan sobre valores proporcionales a la intensidad de la luz.

Como objetivo secundario hay que validar las normales, igual que se hizo con las coordenadas de textura en el Práctico 08: la tupla de vértices tiene normales desde el Práctico 03 y el terreno tiene las que se calcularon en el Práctico 07, pero ningún shader las usó todavía. Este práctico es el primero en el que se puede ver si están bien calculadas.

Punto de partida

La escena del Práctico 08: el terreno texturado, el fondo con el cubemap y el suelo continuado de la Parte 5, y la aeronave armada con primitivas del Práctico 04. No hace falta incorporar ninguna biblioteca nueva. El modelo de iluminación de Phong, el cálculo en coordenadas de mundo, la matriz normal y los tipos de luces son los que se presentaron en las filminas de la clase; la niebla se explica completa en la Parte 4 de este enunciado y la corrección gamma, que se vio en la Unidad III, se resume en la Parte 5.

Parte 1 - La iluminación como código compartido entre shaders

Hasta ahora, en la escena, hay tres objetos que se dibujan con shaders distintos: el terreno, el fondo (que por debajo del horizonte pinta el suelo continuado) y la aeronave. Los tres tienen que estar iluminados por el mismo sol y afectados por la misma niebla; si alguno usa otro modelo u otros valores la diferencia se nota, por ejemplo como una costura en el borde de la ventana de terreno, donde la malla se encuentra con el suelo continuado. Es por esto que conviene que tanto los parámetros de la iluminación como las funciones que la calculan existan una sola vez en el proyecto, y que cada shader las use en lugar de repetirlas.

Responsabilidades

- Los parámetros de la iluminación de la escena (dirección y color del sol, luz ambiente) y los de la niebla (color y visibilidad) se definen en un único lugar del lado del CPU, y desde ahí se envían a todos los programas (shaders) que los usan.
- Las funciones de iluminación y de niebla (y las de la corrección gamma de la Parte 5) se escriben una sola vez en GLSL, en un archivo común, y todos los *fragment shaders* de la escena las usan.
- El material de cada objeto (los coeficientes k_d, k_s y n_s del modelo de Phong) es un dato de cada objeto y no de la escena.

Por dónde empezar

GLSL no tiene una directiva para incluir archivos (no existe un `#include` como en C++), pero la función `glShaderSource(shader, cantidad, cadenas, longitudes)` recibe un arreglo de cadenas y las compila como si fueran un único texto, una a continuación de la otra. Esto permite armar el código fuente de cada *fragment shader* con tres partes: la línea `#version`, el código común y el código propio de ese shader. La línea de versión tiene que ser la primera del texto, por lo que no puede estar dentro de ninguno de los dos archivos y la tiene que agregar el código que arma el programa. La lectura de los archivos ya es responsabilidad del `ResourceManager` y la compilación del módulo `Shader` (Práctico 01), por lo que hay que ampliarlos para que acepten más de un archivo por etapa.

En el siguiente fragmento de código se muestra una propuesta del código común para la iluminación y la niebla. La función de iluminación es la de las filminas de la clase adaptada a una luz direccional (Parte 2) y la de niebla se explica en la Parte 4.

```
// iluminacion.glsl - codigo comun a todos los fragment shaders de la escena.
// Sin #version: esa linea la agrega el codigo que arma el programa.

uniform vec3  uSolDir;        // direccion HACIA el sol, en mundo, normalizada
uniform vec3  uSolColor;      // color e intensidad del sol
uniform vec3  uAmbiente;      // luz ambiente
uniform vec3  uOjo;           // posicion de la camara, en mundo
uniform vec3  uNieblaColor;   // color del horizonte del cielo (Parte 4)
uniform float uVisibilidad;   // distancia a la que la niebla llega al 63 % (Parte 4)

struct Material
{
    vec3  kd;                 // coeficiente difuso (y ambiente: ka = kd)
    vec3  ks;                 // coeficiente especular
    float ns;                 // exponente especular (brillo)
};

// Modelo de Phong con una luz direccional. P y N en coordenadas de mundo, N normalizada.
vec3 iluminar(vec3 P, vec3 N, Material m)
{
    vec3  L     = uSolDir;                        // direccional: la misma en todo punto
    vec3  V     = normalize(uOjo - P);            // hacia el ojo
    float NdotL = max(dot(N, L), 0.0);

    vec3 ambiente  = uAmbiente * m.kd;
    vec3 difusa    = uSolColor * m.kd * NdotL;
    vec3 especular = vec3(0.0);
    if (NdotL > 0.0) {
        vec3 R = reflect(-L, N);
        especular = uSolColor * m.ks * pow(max(dot(R, V), 0.0), m.ns);
    }
    return ambiente + difusa + especular;
}

// Niebla exponencial: mezcla el color con el de la niebla segun la distancia al ojo.
vec3 aplicar_niebla(vec3 color, vec3 P)
{
    float d = length(P - uOjo);
    float f = 1.0 - exp(-d / uVisibilidad);
    return mix(color, uNieblaColor, f);
}
```

Tener en cuenta

- Si el código común declara un `uniform` que el shader propio ya declaraba, el compilador informa un error de redefinición. Puede suceder con `uOjo` en el shader del fondo de la Parte 5 del Práctico 08 (`background-suelo.fs`): hay que dejar una sóla la declaración en el shader.
- Con `glProgramUniform*` cada programa tiene sus propios `uniform`, aunque se llamen igual en los otros shaders, por lo que los valores del sol y de la niebla hay que enviarlos a cada programa por separado. La ubicación de cada `uniform` se obtiene en cada programa con `glGetUniformLocation`.
- El compilador elimina los `uniform` que el shader no usa. Para esos `glGetUniformLocation` devuelve -1, y una llamada a `glProgramUniform*` con ubicación -1 no hace nada y no informa ningún error. Por ejemplo, si un shader no llama a `aplicar_niebla(...)`, en ese programa no existe `uVisibilidad`.
- `uSolDir` es la dirección **hacia** el sol, desde la superficie hacia la luz. Si se la define como la dirección en la que viajan los rayos, el signo queda invertido y se ilumina el lado opuesto de cada superficie.

Decisiones de diseño a tomar

- ¿Dónde se guardan los parámetros de la iluminación de la escena (un `struct` propio, el módulo que dibuja la escena u otro lugar)? ¿Quién los envía a los programas y cada cuánto: una vez al inicio, sólo cuando cambian o en cada cuadro?
- ¿Cómo se arma el código fuente de un shader con más de un archivo: una lista de archivos por etapa, un archivo común fijo que se agrega siempre, u otra forma?
- El código propuesto usa el modelo de Phong, con el vector reflejado R. El modelo de Blinn-Phong, con el vector medio H, es una alternativa válida y cuál usar es decisión que hay que tomar (si se usa Blinn-Phong, el exponente tiene que ser unas cuatro veces mayor para obtener un brillo de tamaño parecido).

Parte 2 - El terreno iluminado por el sol

La luz del sol

El sol está tan lejos de la escena que sus rayos llegan paralelos a todos los puntos: es una *luz direccional*, con la misma dirección L en toda la escena (en la representación con un `vec4` de las filminas en donde $w = 0$). Por lo tanto no hace falta calcular L en cada fragmento como en la luz puntual, sino que es un `uniform`. Para que la escena sea coherente conviene que la dirección elegida coincida con la del sol pintado en el cubemap o, si el cielo no tiene un sol visible, con la dirección desde donde viene la luz en sus imágenes.

Los cambios en los shaders del terreno

El *vertex shader* del terreno de la Parte 2 del Práctico 08 pasa a través del pipeline al *fragment shader* sólo las coordenadas de textura. Para iluminar hacen falta además la posición y la normal de cada fragmento, las dos en coordenadas de mundo (el mismo sistema de coordenadas en el que están el sol y la cámara). El *vertex shader* las tiene que calcular con la matriz de modelo y con la matriz normal (Parte 3) y enviarlas como variables `out`. Al llegar al *fragment shader* esas variables están interpoladas entre los vértices del triángulo, y la normal interpolada ya no es unitaria, por lo que hay que volver a normalizarla.

En el siguiente fragmento de código se muestra una propuesta del *fragment shader* del terreno usando el código común. La textura deja de ser el color final y pasa a ser el coeficiente k_d del material. Como se interpolan los vectores normales y el cálculo de la iluminación se hace por *fragmento*, este ejemplo corresponde a sombreado Phong.

```
// terreno.fs - sin #version: el programa se arma con #version + iluminacion.glsl + este archivo
in vec3 vPosMundo;
in vec3 vNormalMundo;
in vec2 vTexCoord;

layout (binding = 0) uniform sampler2D uTextura;

out vec4 FragColor;

void main()
{
    vec3 N = normalize(vNormalMundo);           // interpolada: hay que volver a normalizar

    Material m;
    m.kd = texture(uTextura, vTexCoord).rgb;    // en el Practico 08 era el color final
    m.ks = vec3(0.0);                           // sin brillo especular
    m.ns = 1.0;

    vec3 color = iluminar(vPosMundo, N, m);
    FragColor = vec4(color, 1.0);
}
```

Tener en cuenta

- Del lado opuesto al sol la componente difusa vale cero y lo único que queda es la ambiente. Si la luz ambiente es muy baja las laderas en sombra quedan negras, y si es muy alta el relieve se aplana.
- La suma de la luz ambiente y la del sol puede sumar más de 1, y en ese caso el color se recorta y aparecen zonas quemadas en blanco. Conviene elegir las intensidades para que sumen cerca de 1 (por ejemplo, 0,3 de ambiente y 0,7 de sol).

Decisión de diseño a tomar

La propuesta deja al terreno sin componente especular. Con valores altos de k_s y n_s toda la superficie brilla de la misma forma y el pasto se ve húmedo o de plástico (el efecto se nota mirando hacia el sol). ¿Qué material le corresponde al terreno? Dejar escrita la elección y por qué.

Cómo hacer la verificación

- Comparar tres imágenes de la escena tomadas desde la misma cámara: con la iluminación apagada (el color es directamente k_d, como en el Práctico 08), con las normales del Práctico 07 y con todas las normales reemplazadas por $(0, 1, 0)$. Con las normales del Práctico 07 el relieve tiene que verse y con la normal constante tiene que desaparecer (salvo en el contorno de las elevaciones contra el cielo) aunque la geometría sea la misma. Esto muestra que lo que se percibe del relieve viene de la normal y no de la posición de los vértices.
- Mover el sol entre una posición alta (cerca del mediodía) y una baja (cerca del atardecer). Con el sol bajo las laderas que miran hacia el sol y las opuestas se diferencian más y el relieve se distingue mejor.
- Si aparecen laderas que miran hacia el sol y están oscuras, revisar el signo de `uSolDir` y el sentido de las normales del Práctico 07: la componente y tiene que ser positiva.

El suelo continuado

El fondo de la Parte 5 del Práctico 08 pinta por debajo del horizonte un plano que continúa el terreno. Para que no se distinga el borde de la ventana de terreno ese plano tiene que iluminarse con la misma función y con el mismo material que la malla. El punto del suelo que se ve en cada píxel ya está calculado en ese shader (es $\vec{P}(t) = \vec{O} + t\,\vec{d}$, con su altura igual a la del plano) y la normal de un plano horizontal es $(0, 1, 0)$, por lo que alcanza con llamar a `iluminar(...)` con esos dos datos y con la textura del suelo como k_d. Por encima del horizonte el cielo no se ilumina, porque el cubemap ya es la

imagen de un cielo iluminado. Si hay relieve cerca del borde de la ventana de terreno, la diferencia de orientación entre la malla y el plano también se ve; es la misma limitación que se describe en la Parte 5 del Práctico 08 y la niebla de la Parte 4 es lo que termina de disimularla.

Parte 3 - La aeronave iluminada

Las piezas de la aeronave se dibujan con su propio shader, que hasta acá pintaba cada pieza con el color declarado en su `RenderItem`. Ese shader tiene que usar el mismo código común que el terreno, con la diferencia de que el material sale de cada pieza: el color del `RenderItem` pasa a ser el coeficiente k_d, y k_s y n_s se pueden fijar para toda la aeronave o por pieza.

En el siguiente fragmento de código se muestra una propuesta para llevar el material de cada pieza hasta el shader. El `struct Material` del lado del CPU tiene los mismos tres campos que el del código común de la Parte 1 y reemplaza al color en el `RenderItem` propuesto en el Práctico 04. En GLSL un `uniform` puede ser de un tipo `struct`, y la ubicación de cada uno de sus campos se obtiene con `glGetUniformLocation` usando el nombre completo del campo, con punto (`uMaterial.kd`).

```
// Material.h - los mismos tres campos que el struct Material de iluminacion.glsl
struct Material {
    glm::vec3 kd = glm::vec3(1.0f);    // el color que antes llevaba el RenderItem
    glm::vec3 ks = glm::vec3(0.0f);
    float     ns = 1.0f;
};

// RenderItem.h - el del Practico 04, con el material en lugar del color
struct RenderItem {
    const Mesh* mesh     = nullptr;
    glm::mat4   model    = glm::mat4(1.0f);
    Material    material;
};

// Al dibujar cada pieza. El shader de la aeronave declara: uniform Material uMaterial;
void enviar_material(GLuint prog, const Material& m)
{
    glProgramUniform3fv(prog, glGetUniformLocation(prog, "uMaterial.kd"), 1, glm::value_ptr(m.kd));
    glProgramUniform3fv(prog, glGetUniformLocation(prog, "uMaterial.ks"), 1, glm::value_ptr(m.ks));
    glProgramUniform1f (prog, glGetUniformLocation(prog, "uMaterial.ns"), m.ns);
}
```

Como el código común va antes que el código propio en el texto que se compila, el `struct` ya está definido cuando el shader de la aeronave declara `uniform Material uMaterial;`, y ese `uniform` se puede pasar directamente a la función de iluminación: `vec3 color = iluminar(vPosMundo, N, uMaterial);`.

Decisión de diseño a tomar

Si en el Práctico 08 se agregó una textura a algunas piezas, el material y la textura tienen que convivir. ¿La textura reemplaza a k_d, como en el terreno, o lo multiplica? ¿El material se fija por pieza, por tipo de pieza o uno solo para toda la aeronave? Esta propuesta no es la única posible: el material puede ser también un dato compartido entre varias piezas, al que el `RenderItem` sólo apunta.

La matriz normal

La posición de cada vértice de una pieza se lleva a coordenadas de mundo con la matriz de modelo M que viene en `RenderItem`, que ya incluye la pose de la aeronave. La normal, en cambio, no se puede transformar con la misma matriz: cuando M tiene un escalado no uniforme, $M\,\bar{n}$ deja de ser perpendicular a la superficie. La matriz que conserva la perpendicularidad es la *matriz normal*, $G = (M^{-1})^T$, calculada con la parte superior izquierda 3×3 de M (una normal es una dirección y la traslación no la afecta); la deducción está en las filminas de la clase. Como sólo depende de M conviene calcularla en el CPU una vez por pieza (`glm::transpose(glm::inverse(glm::mat3(item.model)))`) y enviarla como `uniform` de tipo `mat3`, en lugar de invertir una matriz en cada vértice (en el shader).

Con esto se puede contestar una de las cuestiones del Práctico 04: el error de usar M para transformar las normales no aparece en todas las piezas. Una placa hecha con un cubo escalado sobre sus propios

ejes tiene las normales alineadas con esos ejes, y el escalado sólo les cambia la longitud (que se corrige al normalizar), por lo que la iluminación sale bien aunque la matriz usada esté mal. Lo mismo pasa con el lateral de un cilindro estirado a lo largo de su eje. El error aparece cuando la normal tiene componentes sobre dos ejes que se escalan de forma distinta, por ejemplo en el lateral de un cono estirado a lo largo de su eje o en un cilindro de sección elíptica. Es por esto que una aeronave hecha sólo con placas puede verse bien con la matriz equivocada.

Cómo hacer la verificación

- Las normales de las primitivas se usan por primera vez en este práctico, por lo que conviene verificarlas antes de mirar la iluminación. Una forma directa es pintar cada fragmento con su normal como color, `FragColor = vec4(N * 0.5 + 0.5, 1.0);` (la operación lleva cada componente del intervalo $[-1, 1]$ al $[0, 1]$). Una cara que mira hacia $+x$ se ve rosada, (1; 0,5; 0,5), una que mira hacia $+y$ verde claro y una que mira hacia $+z$ lila (ver el detalle a continuación y la Figura 1).
- Elegir una pieza con normales inclinadas respecto de los ejes de escalado (un cono estirado a lo largo de su eje, por ejemplo) y comparar la iluminación usando M y usando la matriz normal: con M el brillo queda corrido respecto de donde la geometría lo pone.
- Mover la aeronave en distintas actitudes de vuelo y verificar que la cara iluminada de cada pieza sea siempre la que mira hacia el sol.

En la Figura 1 se muestra cómo se ven las normales pintadas como color en una esfera y en cuatro triángulos planos con distintas orientaciones, con la cámara ubicada sobre el eje z y mirando hacia $-z$. En la esfera, el color cambia en forma continua: lila en el centro, donde la normal apunta hacia la cámara ($+z$), rosado en el borde derecho ($+x$) y verde claro en el borde superior ($+y$). Cada triángulo es plano y tiene un único color: lila el del centro, que mira hacia la cámara, verde claro el de abajo, que es horizontal, rosado el de la derecha, que está en un plano de x constante, y celeste el de la izquierda, inclinado 45°, cuya normal (0; 0,71; 0,71) tiene componentes sobre dos ejes.

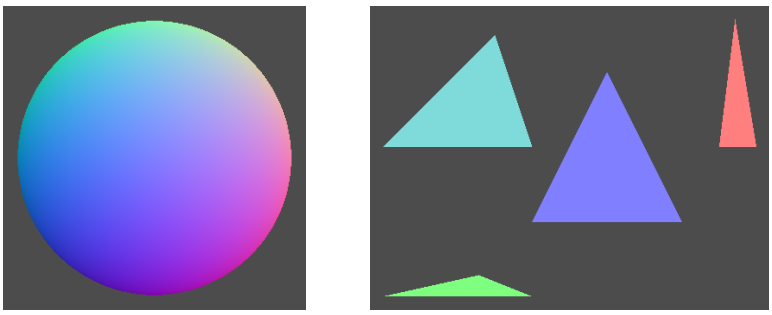


Figura 1: Normales pintadas como color con la cámara mirando hacia $-z$.

Parte 4 - Niebla

El efecto

En la atmósfera la luz que llega desde un objeto lejano atraviesa una capa de aire más gruesa que la que llega desde un objeto cercano y en el camino se mezcla con la luz que dispersan las partículas del aire. Por eso los objetos lejanos se ven con menos contraste y con un color que tiende al del horizonte. La *niebla* en computación gráfica intenta reproducir este efecto de forma simple: el color iluminado de cada fragmento se mezcla con un color fijo (el color de la niebla) en una proporción que depende de la distancia d entre el fragmento y el observador. Además de dar sensación de profundidad, la niebla permite esconder el límite de lo que se dibuja (es una técnica muy usada en videojuegos para dibujar menos polígonos), que en la escena del simulador es el borde de la ventana de terreno.

El factor de mezcla
La mezcla de colores se escribe como

$$C = (1 - f)\, C_{luz} + f\, C_{niebla}$$

donde $f \in [0, 1]$ es el *factor de niebla*: con $f = 0$ el color es igual al color obtenido por iluminación y sombreado (C_{luz}), con $f = 1$ es igual al de la niebla (C_{niebla}). Hay tres formas habituales de calcular f a partir de la distancia:

- **lineal**, $f = (d - d_0)/(d_1 - d_0)$ recortado al intervalo $[0, 1]$: no hay niebla hasta la distancia d_0 y es total desde d_1;
- **exponencial**, $f = 1 - e^{-d/V}$;
- **exponencial cuadrática**, $f = 1 - e^{-(d/V)^2}$, se obtiene niebla más transparente cerca del observador y más cerrada en la lejanía.

Se propone usar la exponencial, que es la que calcula la función `aplicar_niebla(...)` de la Parte 1. El parámetro V es la *visibilidad*, una distancia que se elige: a $d = V$ el color es en un 63 % el de la niebla, a $2V$ en un 86 % y a $3V$ en un 95 %. La distancia se calcula en coordenadas de mundo, con la posición del fragmento y la de la cámara que ya usa la iluminación.

La niebla se aplica **después** de la iluminación, porque es un color que no depende de la orientación de la superficie ni de la posición del sol (la niebla no se ilumina). En el *fragment shader* del terreno de la Parte 2 alcanza con agregar la línea `color = aplicar_niebla(color, vPosMundo);` antes de escribir `FragColor`. Como comentario final, la función `mix` de GLSL implementa exactamente la ecuación de mezcla presentada antes.

El color de la niebla
Si se usa un color fijo cualquiera (un gris, por ejemplo) aparece una franja sobre el horizonte, porque el terreno lejano se funde con el gris pero el cielo no recibe niebla. Para que la lejanía se funda con el cielo, el color de la niebla tiene que ser el del horizonte del cubemap. Se puede determinar una sola vez, en el CPU, al cargar las imágenes del cielo como el promedio de los píxeles de la fila central de las cuatro caras laterales del cubemap (que es la fila que corresponde al horizonte, elevación cero). Las componentes del promedio se dividen por 255 para llevarlas al intervalo $[0, 1]$.

Dónde se aplica

- En el terreno, usando la posición de cada fragmento.
- En el suelo continuado, usando el punto $\vec{P}(t)$ del plano. Como t crece sin límite hacia el horizonte, f tiende a 1 y el suelo llega al horizonte con el color de la niebla.
- En la aeronave, usando la posición de cada fragmento. Desde las cámaras cercanas el efecto casi no se nota, pero con una cámara alejada la aeronave también se funde con la lejanía.
- En el cielo, sólo para los fragmentos cerca del horizonte. Las direcciones del cubemap no tienen una distancia asociada, pero conviene aplicar la mezcla al color del cielo en los primeros grados de elevación para que a elevación cero, el cielo y el suelo tengan el mismo color. Tomando la dirección $\vec{d}$ normalizada, d_y es el seno de la elevación, entonces, el factor f calculado como `f = 1.0 - smoothstep(0.0, 0.10, d.y)` hace la transición en los primeros 5,7° aproximadamente.

La niebla y el tamaño de la ventana de terreno
La Parte 5 del Práctico 08 dejó planteado que el suelo continuado repite la textura pero no la forma del terreno, por lo que si el relieve tiene variaciones importantes cerca del borde de la ventana de terreno, se distingue el cambio a un plano infinito. La niebla lo resuelve si en el borde el factor f está cerca de 1. Con la cámara en el centro de una ventana de terreno de lado L, el borde más cercano está a una distancia $L/2$ y para que ahí la niebla sea del 92 %, hace falta que la visibilidad V en el shader sea

$$\frac{L}{2} \geq 2{,}5\, V$$

Si la ventana tiene 2 [km] de lado, la visibilidad puede ser de 400 [m] como máximo, que es una niebla

muy cerrada para un simulador de vuelo; para una visibilidad de 4 [km] la ventana tiene que tener 20 [km] de lado.

Tener en cuenta

- La distancia que usa la niebla es la distancia en el espacio, no la horizontal. A una altitud h el punto del terreno más cercano está, como mínimo, a una distancia h, por lo que si la altitud es comparable a la visibilidad V, el terreno entero queda cubierto por la niebla (esto es algo que se puede ajustar cambiando la visibilidad como una función de la altitud).
- El suelo continuado se dibuja con el shader del fondo (cielo), por lo que necesita el color de la niebla y la visibilidad igual que el terreno.

Decisión de diseño a tomar

¿Qué visibilidad y qué tamaño de ventana de terreno? Una ventana más grande permite más visibilidad pero tiene más vértices (la tabla de memoria del Práctico 07); si el relieve cerca del borde es suave, el suelo continuado puede alcanzar con una niebla más abierta. También hay que tener en cuenta la altitud a la que se va a volar. Dejar escritos los valores elegidos y por qué.

Cómo hacer la verificación

- Reemplazar el color de la niebla por un gris (u otro color que resalte) y verificar que aparece la franja sobre el horizonte; con el color del horizonte del cubemap tiene que desaparecer.
- Mirar hacia el borde de la ventana de terreno, en una zona con relieve, con y sin niebla: con niebla no se tiene que distinguir dónde termina la malla.
- Subir con la cámara y anotar a qué altitud el terreno empieza a perderse en la niebla con la visibilidad elegida.

Parte 5 - Corrección gamma

Las ecuaciones del modelo de iluminación (Partes 2 y 3) y de la niebla (Parte 4) suman y multiplican intensidades de luz; esto es válido sólo si los valores con los que se opera son proporcionales a la intensidad, es decir, si están en una escala lineal. Sin embargo, como se vio en la Unidad III, la pantalla no es lineal: un valor v entre 0 y 1 se muestra con una intensidad aproximadamente igual a $v^{2,2}$. Es por esto que las imágenes (las texturas y el cubemap) y los colores elegidos a ojo vienen codificados en sRGB, con la curva inversa ($v^{1/2,2}$), de manera que se vean bien en la pantalla sin ninguna conversión.

Si el resultado de la iluminación se escribe tal cual, la pantalla le aplica la curva y la iluminación queda distorsionada. Por ejemplo, una superficie que recibe la mitad de la luz del sol ($N \cdot L = 0{,}5$) se escribe como 0,5 y la pantalla la muestra con una intensidad de $0{,}5^{2,2} \approx 0{,}22$, por lo que el paso de la luz a la sombra resulta más brusco de lo que corresponde. La *corrección gamma* consiste en hacer las cuentas en lineal y convertir el resultado a sRGB antes de escribirlo: en el ejemplo se escribe $0{,}5^{1/2,2} \approx 0{,}73$, y la pantalla lo muestra con una intensidad de 0,5.

Las dos conversiones

La corrección requiere dos pasos en cada *fragment shader*: pasar a lineal cada color que entra en las cuentas y, al final de todo (después de la niebla), pasar el resultado a sRGB. En el siguiente fragmento de código se muestran las dos funciones de conversión, que se agregan al código común `iluminacion.glsl` de la Parte 1. Se usa la aproximación de la curva sRGB con $\gamma = 2{,}2$ que se presentó en la Unidad III.

```
// Agregado a iluminacion.glsl: conversiones entre sRGB y lineal (Parte 5).
const float GAMMA = 2.2;

vec3 a_lineal(vec3 c) { return pow(c, vec3(GAMMA)); }        // color en sRGB -> lineal
vec3 a_srgb(vec3 c)   { return pow(c, vec3(1.0 / GAMMA)); }  // resultado lineal -> sRGB
```

Con estas funciones, el `main()` del *fragment shader* del terreno de la Parte 2, con la niebla de la Parte 4 ya agregada, queda como se muestra en el siguiente fragmento de código (las declaraciones que están antes de `main()` no cambian).

```
void main()
{
    vec3 N = normalize(vNormalMundo);

    Material m;
    m.kd = a_lineal(texture(uTextura, vTexCoord).rgb);  // la textura esta en sRGB
    m.ks = vec3(0.0);
    m.ns = 1.0;

    vec3 color = iluminar(vPosMundo, N, m);             // las cuentas, en lineal
    color = aplicar_niebla(color, vPosMundo);           // uNieblaColor ya en lineal
    FragColor = vec4(a_srgb(color), 1.0);               // al final, vuelta a sRGB
}
```

Tener en cuenta

- Para saber qué valores hay que pasar a lineal no importa si salen de una imagen o se escriben a mano, sino qué representan. Un color que se eligió por cómo se ve en la pantalla está en sRGB, porque es el valor que, escrito sin ninguna conversión, produce ese aspecto. Es el caso de las texturas, del cubemap y también del k_d de las piezas de la aeronave, que se eligió en el Práctico 04 mirando el resultado en la pantalla sin corrección. Si ese k_d no se pasa a lineal (en el shader o una sola vez en el CPU, al armar el material), con la corrección activa la pieza se ve más clara y desaturada que en el Práctico 04.
- Las intensidades `uSolColor` y `uAmbiente`, en cambio, no se eligen por su aspecto sino como proporciones de luz que se suman (0,3 de ambiente y 0,7 de sol suman la luz total, en la propuesta de la Parte 2), por lo que ya son valores lineales y no se convierten. Lo mismo vale para k_s, que es la fracción de la luz que se refleja como brillo.
- El cielo no se ilumina, pero su color pasa igual por la conversión final a sRGB, por lo que si antes no se lo pasa a lineal se ve más claro y desaturado (lavado).
- El color de la niebla se calcula en el CPU como un promedio de píxeles de las imágenes del cubemap (Parte 4), por lo que también está en sRGB. Como `aplicar_niebla(...)` lo usa tal cual, conviene pasarlo a lineal antes de enviarlo a los shaders, por ejemplo con `glm::pow(color, glm::vec3(2.2f))`.
- La verificación de las normales de la Parte 3 escribe la normal como color sin convertirla, porque es un dato (una dirección codificada como color) y no un color.
- Una superficie que mira de frente al sol, con la luz ambiente y la del sol sumando 1, se ve con el color de la textura, igual que sin la corrección. Lo que cambia es la transición hacia la sombra, que se vuelve más gradual, y el lado opuesto al sol, que se ve más claro (con la luz ambiente en 0,3, la cara en sombra se ve con el 58 % del color de la textura en lugar del 30 %). Por lo tanto, la luz ambiente de la Parte 2 y la visibilidad de la Parte 4 hay que volver a ajustarlas con la corrección activa.

Decisión de diseño a tomar

Las dos conversiones las puede hacer también el hardware, sin código en el shader y con la curva sRGB exacta en lugar de la aproximación con 2,2. Una textura creada con el formato interno `GL_SRGB8_ALPHA8` (en lugar de `GL_RGBA8`) devuelve al muestrearla el color ya convertido a lineal, y con `glEnable(GL_FRAMEBUFFER_SRGB)` OpenGL convierte a sRGB cada color que se escribe en el framebuffer. Para esta segunda conversión el framebuffer de la ventana tiene que admitirla, lo que depende de la configuración de GLFW (la opción `GLFW_SRGB_CAPABLE`) y del controlador de video, por lo que hay que verificarlo en la máquina en la que se trabaja. ¿Qué camino usar? Cualquiera de los dos es válido, pero si un mismo color se convierte en el shader y además en el hardware, la conversión se aplica dos veces y la imagen se ve lavada (o demasiado oscura, si lo que se repite es la conversión a lineal).

Cómo hacer la verificación

- Conviene poder activar y desactivar la corrección durante la ejecución (por ejemplo, con un `uniform bool` que cambie con una tecla) para comparar la misma vista con y sin ella. La diferencia se nota más sobre las piezas de la aeronave con brillo especular que sobre el terreno, donde la textura disimula el cambio.
- El cielo tiene que verse igual con y sin la corrección. Si con la corrección se ve más pálido, falta pasar el color del cubemap a lineal.
- Con la corrección activa tampoco tiene que aparecer la franja sobre el horizonte de la Parte 4. Si aparece, falta pasar el color de la niebla a lineal.

Opcional - Ciclo día/noche

Para lograr un ciclo día/noche la dirección del sol puede variar con el tiempo de la simulación: girando alrededor de un eje horizontal, el sol sale, sube hasta el mediodía, baja y se pone, y con eso la iluminación de la escena cambia sola. Para que el efecto sea creíble hacen falta algunos ajustes además de la dirección:

- El color del sol se vuelve más cálido (anaranjado) cuando el sol está cerca del horizonte.
- Cuando el sol está por debajo del horizonte hay que apagarlo. El modelo de Phong no tiene sombras (cada punto se ilumina sin tener en cuenta a los demás), por lo que las caras inferiores de la aeronave quedarían iluminadas a través del terreno.
- La luz ambiente y el cielo tienen que oscurecerse. El cubemap es una imagen de un cielo de día, por lo que hay que multiplicar su color por un factor que dependa de la hora, y el color de la niebla con él.

Cómo se sabe que está bien

- Con la iluminación encendida se ve el relieve del terreno y con la normal constante desaparece.
- Las laderas que miran hacia el sol son las más claras y ninguna superficie se ilumina del lado opuesto al sol.
- Pintadas con su normal como color, todas las primitivas muestran colores coherentes con su orientación.
- La iluminación de cada pieza de la aeronave sigue al sol con cualquier actitud de vuelo, también en las piezas con escalado no uniforme.
- No se distingue el borde entre la malla del terreno y el suelo continuado, ni por la luz ni por la niebla.
- No hay una franja de color entre el terreno lejano y el cielo.
- Con la corrección gamma activa el cielo se ve igual que sin ella, y la transición entre las caras iluminadas y las caras en sombra es más gradual.
- Están escritos el material del terreno, la visibilidad y el tamaño de la ventana de terreno elegidos (con la corrección gamma activa), y la altitud a la que el terreno se pierde en la niebla.

Como en los prácticos anteriores, las interfaces y los fragmentos de código mostrados son un **punto de partida y no el molde**. Las decisiones de adentro son de cada uno y hay que poder explicarlas.

El *normal mapping* y la reflexión del cielo sobre las superficies, que aparecen en las filminas de la clase, no forman parte de este práctico. Quedan como mejoras posibles de la apariencia del proyecto y se pueden consultar en particular.

La **presentación de avance del proyecto** está planificada para el **viernes 27 de noviembre**.